

Semester& Year: 3rd Sem & 2025

Faculty Name: Dr. Arpitha Shankar S I

Subject with Code: Object Oriented Programming with JAVA (BCS306A)

Course Outcomes: At the end of the course, the students will be able to:

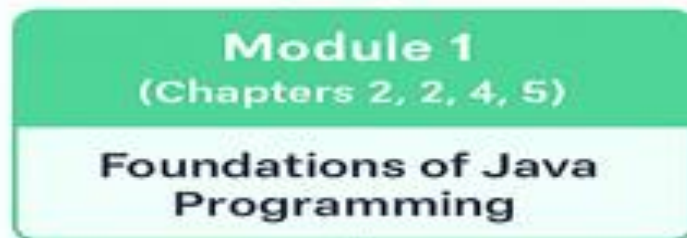
CO1: Demonstrate proficiency in writing simple Java programs involving branching, looping, arrays, and operators using appropriate control structures.

CO2: Design and implement classes with attributes, methods, constructors, and access control for solving given scenarios.

CO3: Apply the concepts of inheritance, polymorphism, and interfaces to develop object-oriented solutions for real-world problems.

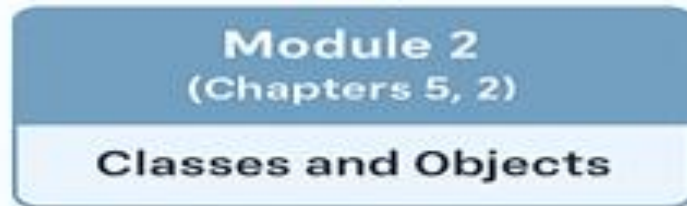
CO4: Utilize packages and implement exception-handling mechanisms to solve complex programming problems.

CO5: Develop multithreaded programs and employ concepts of enumerations, autoboxing, and wrapper classes in program development.



What you'll study:

- Basics of Java syntax and structure
- Variables data types arrays
- Operators for calculations and decisions
- Control statements (if, switch, loops, break, continue)



What you'll study:

- How to define classes and create objects
- Methods, constructors, recursion
- Access modifiers (public, private, protected)
- Special keywords (this, static, final)

After this module; you can write progs, like:

- A Student class with attributes (name, marks) and methods (calculateGrade)
- A BankAccount class with deposit and withdraw methods
- A Stack class implementing push(pop-operations)
- Recursive factorial or Fibonacci series



What you'll study:

- Inheritance
- super keyword and constructor chaining
- Method overriding & runtime polymorphism
- Abstract classes and the Object class
- Interfaces with default, static, and private methods

Module 4 (Chapters 9, 10)

Packages and Exception Handling



Module 5 (Chapters 11, 12)

Advanced Java Concepts

+ By the end of the course

You'll be able to design real-world object-oriented applications in Java on a need-to-know basis.

After this module, you can write programs like:

- A Shape class extended by Circle, Triangle, Square (polymorphism demo)
- A Vehicle superclass with Car and Bike subclasses
- An interface Resizable with Rectangle implementing resize methods
- Demonstrating abstract classes for

After you'll require, you can write programs like:

- A multithreaded program where one thread prints numbers and another prints alphabets
- A program to simulate a countdown timer using threads
- Using enum for days of the week or traffic light states

Suggested Learning Resources:

Textbook

1. Java: The Complete Reference, Twelfth Edition, by Herbert Schildt, November 2021, McGraw-Hill, ISBN: 9781260463422

Reference Books

1. Programming with Java, 6th Edition, by E Balagurusamy, Mar-2019, McGraw Hill Education, ISBN: 9789353162337.
2. Thinking in Java, Fourth Edition, by Bruce Eckel, Prentice Hall, 2006
(https://sd.blackball.lv/library/thinking_in_java_4th_edition.pdf)

Web links and Video Lectures (e-Resources):

- Java Tutorial: <https://www.geeksforgeeks.org/java/>
- Introduction To Programming In Java (by Evan Jones, Adam Marcus and Eugene Wu):
<https://ocw.mit.edu/courses/6-092-introduction-to-programming-in-java-january-iap-2010/>
- Java Tutorial: <https://www.w3schools.com/java/>
- Java Tutorial: <https://www.javatpoint.com/java-tutorial>

Module-1

An Overview of Java: Object-Oriented Programming (Two Paradigms, Abstraction, The Three OOP Principles), Using Blocks of Code, Lexical Issues (Whitespace, Identifiers, Literals, Comments, Separators, The Java Keywords).

Data Types, Variables, and Arrays: The Primitive Types (Integers, Floating-Point Types, Characters, Booleans), Variables, Type Conversion and Casting, Automatic Type Promotion in Expressions, Arrays, Introducing Type Inference with Local Variables.

Operators: Arithmetic Operators, Relational Operators, Boolean Logical Operators, The Assignment Operator, The ? Operator, Operator Precedence, Using Parentheses.

Control Statements: Java's Selection Statements (if, The Traditional switch), Iteration Statements (while, do-while, for, The For-Each Version of the for Loop, Local Variable Type Inference in a for Loop, Nested Loops), Jump Statements (Using break, Using continue, return).

Chapter 2, 3, 4, 5

History of Java

- Java was developed by **James Gosling** and team at Sun Microsystems in **1991**.
- Initially called “**Oak**”, named after a tree outside Gosling’s office.
- Later renamed to **Java** in **1995** (trademark issues with “Oak”).
- Acquired by **Oracle** in 2010.
- Became popular because of **WORA – Write Once, Run Anywhere**.
- Java is a high-level, object-oriented programming language

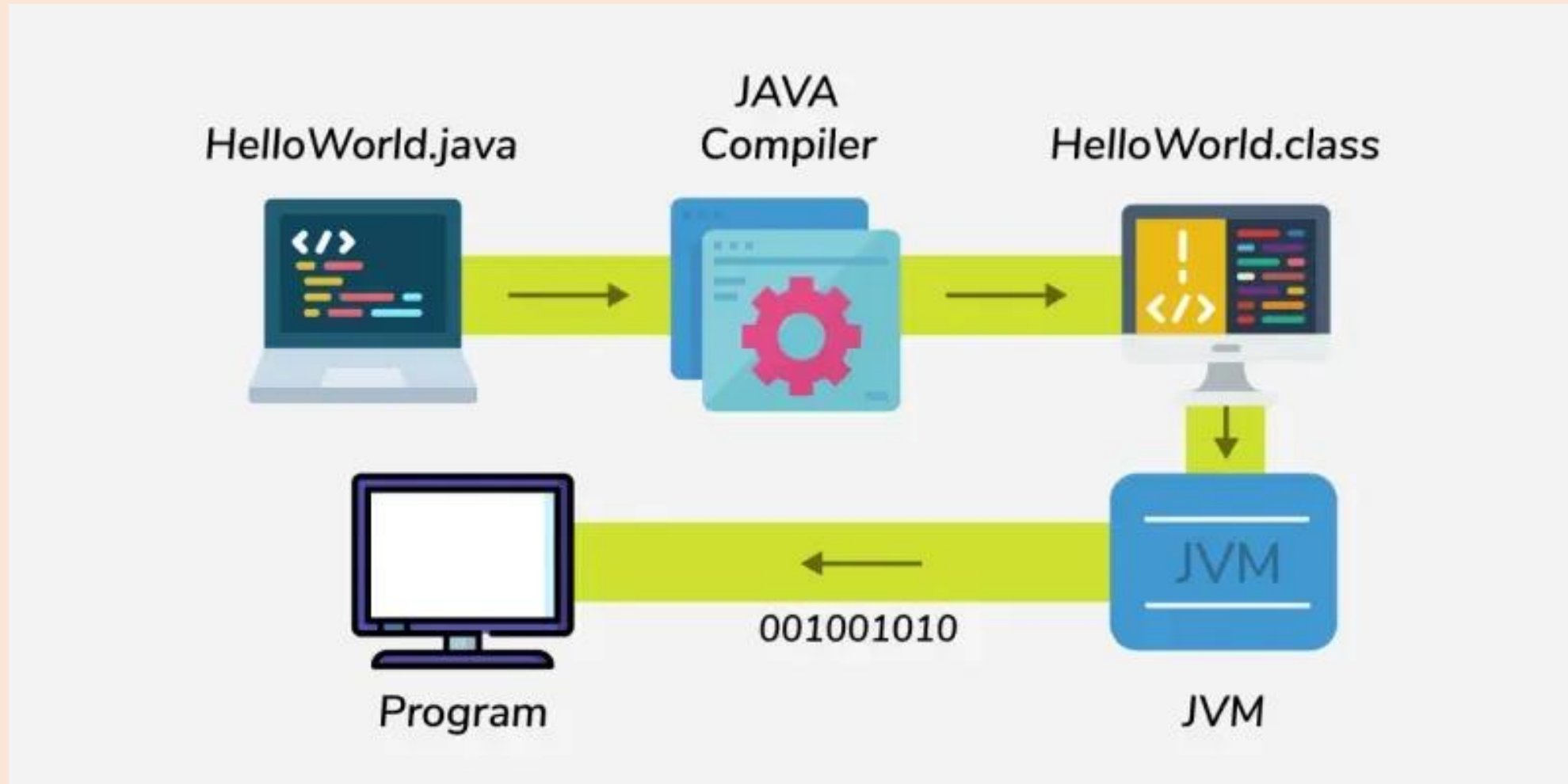
Why Coffee Cup Symbol? ☕

- The name “Java” comes from a type of **coffee from Indonesia (Java coffee)**.
- The team drank lots of coffee while coding → Inspired the logo.
- Coffee cup ☕ = **energy, simplicity, and productivity**.
- Symbol shows Java is a language for **everyday use** like coffee is for daily life.

Features of Java

- Simple and easy to learn
- Object-Oriented
- Platform Independent
- Secure and Robust
- Multithreaded
- High Performance (JVM & JIT)
- Rich API & Community Support

Java program execution follows this below simple flow:



Applications of Java

- Android Mobile Applications
- Web Applications (E-commerce, Banking)
- Desktop GUI Applications
- Enterprise Applications (ERP, CRM)
- Scientific Applications (NASA WorldWind)
- Gaming (Minecraft)
- Big Data & Cloud Computing

Famous Applications Built Using Java

1. Android Apps: Most of the Android mobile apps are built using Java.

example: WhatsApp, Instagram, Paytm

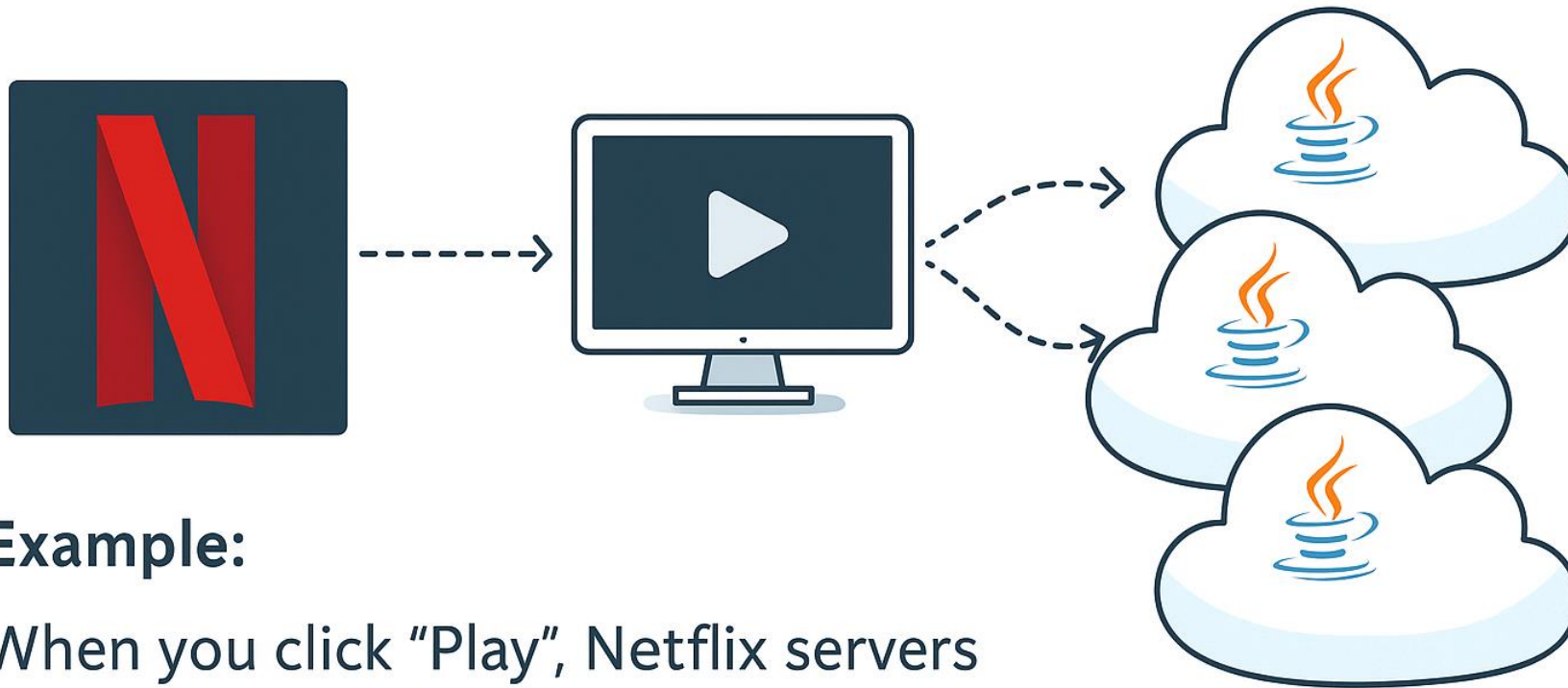
2. Netflix: This uses Java for content delivery and backend services.

For **backend services** like streaming, content delivery, and recommendations.

- Example:**

- When you click “*Play*”, Netflix servers (many built in Java) decide from which server to stream the movie to you.

Netflix: This uses Java for content delivery and backend services.



Example:

When you click "Play", Netflix servers (many built in Java) decide from which server to stream the movie to you.

3. Amazon: Java language is used for its backend systems.



Example: When you add items to your cart and proceed to checkout, Java programs handle these transactions securely and quickly.



5. Spotify: This uses Java in parts of its server-side infrastructure.

4. LinkedIn: Handles millions of user requests and connections.



Example: When you log in and see your job feed or connection Java is working in the background to fetch and deliver data efficiently.



UBER

Java is used for backend services like trip management.

Example: When you ride, Java handles matching you with a nearby driver, tracking the trip, and calculating

3. **Amazon:** Java language is used for its backend systems. (shopping cart, payments, inventory).

•**Example:** When you add items to your cart and proceed to checkout, **Java programs** handle these transactions securely and quickly.

4. LinkedIn

Use of Java: Handles millions of user requests and connections.

Example:

When you log in and see your job feed or connection suggestions, Java is working in the background to fetch and deliver data efficiently.

•**Minecraft:** This is one of the world's most popular games that is built in Java.

5. Spotify: This uses Java in parts of its server-side infrastructure.

- **Use of Java:** Server-side infrastructure.
- **Example:** When you search for a song, Java programs quickly find results from a massive music database.
- Java helps Spotify **scale** as millions listen to music at the same time.

6. Uber: Java is used for backend services like trip management.

- **Use of Java:** For backend trip management.
- **Example:** When you book a ride, Java handles **matching you with a nearby driver**, tracking the trip, and calculating fares.

Java = everywhere → from the app in your phone (Android)
to the ride you book (Uber),
to the movie you stream (Netflix),
to the game you play (Minecraft).

Two programming paradigms

1. Procedure-Oriented Programming (POP)

Focuses on **functions/procedures**.

Data is secondary, functions act on data.

Example: **C language**.

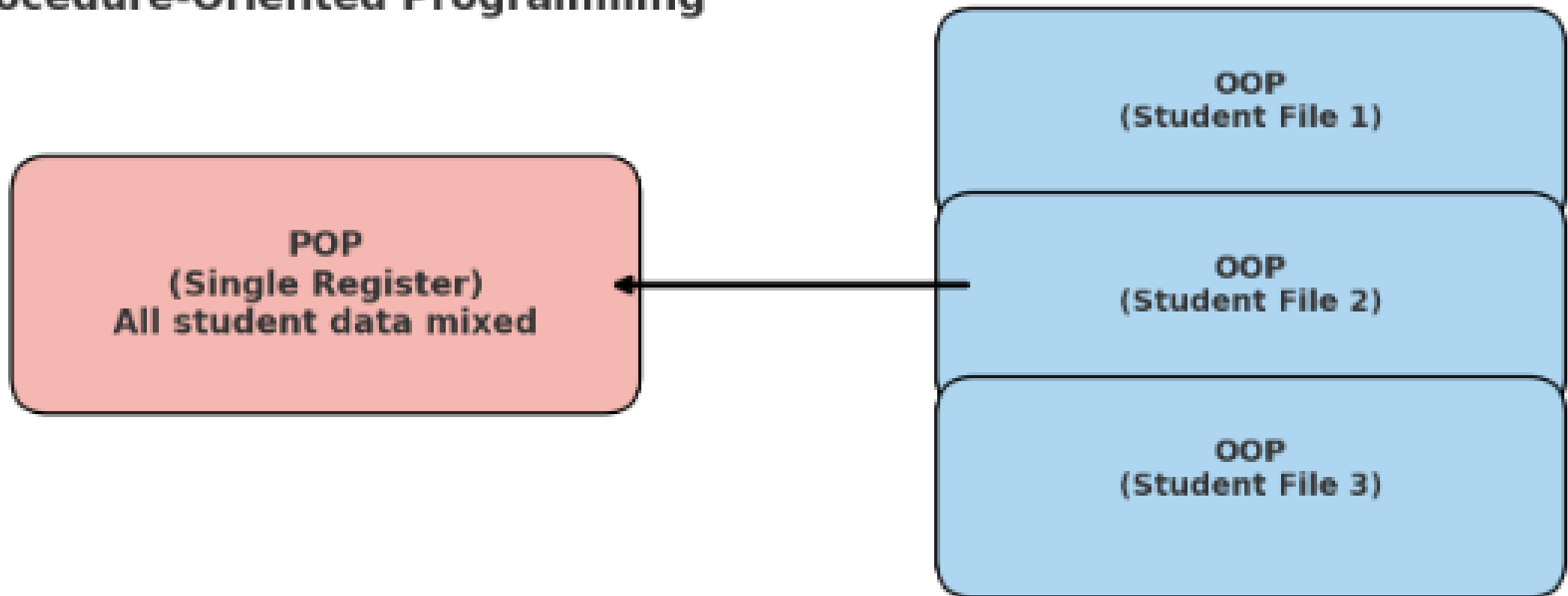
2. Object-Oriented Programming (OOP)

- Focuses on **objects** (data + methods bundled together).
- Data is primary, functions (methods) belong to objects.
- Example: **Java, Python, C++**.

POP vs OOP Analogy: School Records

Object-Oriented Programming

Procedure-Oriented Programming



Procedure-Oriented Programming (POP):

- Imagine a school keeps student information in **different registers**.
 - One register for **names**
 - One register for **marks**
 - One register for **addresses**
- If the teacher wants to find all details about one student, they must **look in each register separately**.
- Functions (procedures) are written to fetch data from these registers.
- Data is **scattered and separate from functions**.

Object-Oriented Programming (OOP):

- Now imagine the school keeps a **student file** for each student.
- Inside each file, everything is stored together:
 - Name
 - Marks
 - Address
 - Attendance
- If the teacher wants details, they just **open the student's file**.
- Each file is like an **object** that contains **data (marks, address)** and **methods (calculate average, print report card)**.
- Data and functions are **bundled together**.

Abstraction

👉 Definition:

Showing only **essential details** and hiding the internal complexity.

👉 Car:

- When you drive a **car**, you use the **steering, brake, accelerator**.
- You don't need to know *how the engine works inside*.
- Car = abstraction (hides engine complexity, shows only what's needed).

Three Main OOP Principles

1. Encapsulation (Data Hiding)

- **Meaning:** Wrapping up **data (variables)** and **methods (functions)** into a single unit (class).
- It also means **restricting direct access** to some details and only exposing what is necessary.
- Achieved using **access modifiers** (private, public, protected)

2. Inheritance (Reusability)

One class (child) can **inherit properties** from another (parent).

Avoids rewriting code.

👉 **Family:** A child inherits traits from parents (like eye color).

3. Polymorphism (Many Forms)

- Allows the same method or operator to behave **differently** based on the situation.

Teacher: same person can play different roles — before exam they are a **mentor**, during exam they are an **examiner**, after school they are a **counselor**.

The same name (“teacher”) has many behaviors based on context.

can act as a **guide**, **examiner**, **mentor** depending on the situation.

A First Simple Program

```
/*  
  
This is a simple Java program.  
  
*/  
  
class Example {  
    public static void main(String args[]) {  
        System.out.println("This is a simple Java program.");  
    }  
}
```

Using Blocks of Code

```
if (x < y) { // begin a block  
x = y;  
y = 0;  
} // end of block
```

Lexical Issues

- Java programs are a collection of whitespace, identifiers, literals, comments, operators, separators, and keywords (Atomic elements of Java)

Whitespace

- Java is a free-form language. This means that you do not need to follow any special indentation rules.
- In Java, whitespace is a space, tab, or newline.

Identifiers

- Identifiers are used to name things, such as classes, variables, and methods.
- An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters.
- Java is case-sensitive, so **VALUE** is a different identifier than **Value**.
- Some examples of valid identifiers are

AvgTemp	count	a4	\$test	this_is_ok
---------	-------	----	--------	------------

Invalid identifier names include these:

2count	high-temp	Not/ok
--------	-----------	--------

Literals

A constant value in Java is created by using a *literal* representation of it.

For example, here are some literals:

100	98.6	'X'	"This is a test"
-----	------	-----	------------------

Left to right, the first literal specifies an integer, the next is a floating-point value, the third is a character constant, and the last is a string.

A literal can be used anywhere a value of its type is allowed.

Comments: There are three types of comments defined by Java.

Single-line

Starts with //

Extends only till the end of that line.

Used for short notes inside code.

Multiline

Starts with /* and ends with */.

Can span across multiple lines.

Used for larger explanations or temporarily removing (commenting out) blocks of code.

Documentation comment.

This type of comment is used to produce an HTML file that documents your program.

The documentation comment begins with a /** and ends with a */.

Separators

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.
::	Colons	Used to create a method or constructor reference. (Added by JDK 8)

The Java Keywords

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Data Types, Variables, and Arrays:

Primitive Types:

- Every variable has a type, every expression has a type, and every type is strictly defined.
- All assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility.
- There are no automatic coercions or conversions of conflicting types as in some languages. The Java compiler checks all expressions and parameters to ensure that the types are compatible.
- Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

Java defines eight *primitive* types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **Boolean**

- **Integers** This group includes **byte**, **short**, **int**, and **long**, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes **float** and **double**, which represent numbers with fractional precision.
- **Characters** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes **boolean**, which is a special type for representing true/false values.

Integers

- Java defines four integer types: **byte**, **short**, **int**, and **long**.
- All of these are signed, positive and negative values.
- Java does not support unsigned, positive-only integers.

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

```
// Compute distance light travels using long variables.
class Light {
    public static void main(String args[]) {
        int lightspeed;
        long days;
        long seconds;
        long distance;

        // approximate speed of light in miles per second
        lightspeed = 186000;

        days = 1000; // specify number of days here

        seconds = days * 24 * 60 * 60; // convert to seconds

        distance = lightspeed * seconds; // compute distance

        System.out.print("In " + days);
        System.out.print(" days light will travel about ");
        System.out.println(distance + " miles.");
    }
}
```

In 1000 days light will travel about 16070400000000 miles.

Floating-Point Types

- Floating-point numbers, also known as *real* numbers, are used when evaluating expressions that require fractional precision.
- There are two kinds of floating-point types, **float** and **double**, which represent single- and double-precision numbers, respectively

Name	Width in Bits	Approximate Range
double	64	4.9e−324 to 1.8e+308
float	32	1.4e−045 to 3.4e+038

float

- The type **float** specifies a *single-precision* value that uses 32 bits of storage.
- Variables of type **float** are useful when you need a fractional component, but don't require a large degree of precision.
- For example, **float** can be useful when representing dollars and cents.
- float hightemp, lowtemp;

double

- Double precision, as denoted by the **double** keyword, uses 64 bits to store a value.
- Double precision is actually faster than single precision on some modern processors that have been optimized for high-speed mathematical calculations.
- All transcendental math functions, such as **sin()**, **cos()**, and **sqrt()**, return **double** values.
- When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, **double** is the best choice.


```
// Compute the area of a circle.  
  
class Area {  
  
    public static void main(String args[]) {  
  
        double pi, r, a;  
  
        r = 10.8; // radius of circle  
  
        pi = 3.1416; // pi, approximately  
  
        a = pi * r * r; // compute area  
  
        System.out.println("Area of circle is " + a);  
  
    }  
  
}
```

Characters

- In Java, the data type used to store characters is char.
- In C/C++, char is 8 bits wide. This is not the case in Java. Instead, Java uses Unicode to represent characters.
- Unicode defines a fully international character set that can represent all of the characters found in all human languages

```
// Demonstrate char data type.
```

```
class CharDemo {  
  
    public static void main(String args[]) {  
  
        char ch1, ch2;  
  
        ch1 = 88; // code for X  
  
        ch2 = 'Y';  
  
        System.out.print("ch1 and ch2: ");  
  
        System.out.println(ch1 + " " + ch2);  
  
    }  
  
}
```

This program displays the following output:

ch1 and ch2: X Y

Booleans

- Java has a primitive type, called boolean, for logical values.
 - It can have only one of two possible values, true or false.
 - This is the type returned by all relational operators, as in the case of $a < b$.
- boolean is also the type required by the conditional expressions that govern the control statements such as if and for.

Variables

The variable is the basic unit of storage in a Java program.

A variable is defined by the combination of an identifier, a type, and an optional initializer.

Declaring a Variable

In Java, all variables must be declared before they can be used.

`type identifier [ = value ][, identifier [= value ] ...];`

`int a, b, c; // declares three ints, a, b, and c.`

`int d = 3, e, f = 5; // declares three more ints, initializing
// d and f.`

`byte z = 22; // initializes z.`

`double pi = 3.14159; // declares an approximation of pi.`

`char x = 'x'; // the variable x has the value 'x'.`

Dynamic Initialization

Although the preceding examples have used only constants as initializers, Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

For example, here is a short program that computes the length of the hypotenuse of a right triangle given the lengths of its two opposing sides:

```
// Demonstrate dynamic initialization.
```

```
class DynInit {  
  
    public static void main(String args[]) {  
  
        double a = 3.0, b = 4.0;  
  
        // c is dynamically initialized  
  
        double c = Math.sqrt(a * a + b * b);  
  
        System.out.println("Hypotenuse is " + c);  
  
    }  
}
```

The Scope and Lifetime of Variables

// Demonstrate block scope.

```
class Scope {  
    public static void main(String args[]) {  
        int x; // known to all code within main  
        x = 10;  
  
        if(x == 10) {  
            int y = 20; // known only to this block  
            // x and y both known here.  
            System.out.println("x and y: " + x + " " + y);  
            x = y * 2;  
        }  
        // y = 100; // Error! y not known here  
        // x is still known here.  
        System.out.println("x is " + x);  
    }  
}
```

```
// Demonstrate lifetime of a variable.
class LifeTime {
public static void main(String args[]) {
    int x;
    for(x = 0; x < 3; x++) {
        int y = -1; // y is initialized each time block is entered
        System.out.println("y is: " + y); // this always prints -1
        y = 100;
        System.out.println("y is now: " + y);
    }
}
}
```

The output generated by this program is shown here:

```
y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100
```


Although blocks can be nested, you cannot declare a variable to have the same name as one in an outer scope. For example, the following program is illegal:

```
// This program will not compile
```

```
class ScopeErr {  
public static void main(String args[]) {  
int bar = 1;  
{ // creates a new scope  
int bar = 2; // Compile-time error – bar already defined!  
}  
  
}  
}
```

Type Conversion and Casting

- It is fairly common to assign a value of one type to a variable of another type.
- If the two types are compatible, then Java will perform the conversion automatically.
- For example, it is always possible to assign an **int** value to a **long** variable.
- However, not all types are compatible, and thus, not all type conversions are implicitly allowed.
- For instance, there is no automatic conversion defined from **double** to **byte**.
- Fortunately, it is still possible to obtain a conversion between incompatible types.
- To do so, you must use a *cast*, which performs an explicit conversion between incompatible types.

Java's Automatic Conversions

When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a *widening conversion* takes place.

For example, the **int** type is always large enough to hold all valid **byte** values, so no explicit cast statement is required.

For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other. However, there are no automatic conversions from the numeric types to **char** or **boolean**. Also, **char** and **boolean** are not compatible with each other.

Casting Incompatible Types

For example, if you want to assign an **int** value to a **byte** variable? This conversion will not be performed automatically, because a **byte** is smaller than an **int**. This kind of conversion is sometimes called a *narrowing conversion*, since you are explicitly making the value narrower so that it will fit into the target type.

To create a conversion between two incompatible types, you must use a cast. A *cast* is simply an explicit type conversion. It has this general form:

(target-type) value

```
int a;
```

```
byte b;
```

```
// ...
```

```
b = (byte) a;
```

The following program demonstrates some type conversions that require casts:

```
// Demonstrate casts.
class Conversion {
    public static void main(String args[]) {
        byte b;
        int i = 257;
        double d = 323.142;

        System.out.println("\nConversion of int to byte.");
        b = (byte) i;
        System.out.println("i and b " + i + " " + b);

        System.out.println("\nConversion of double to int.");
        i = (int) d;
        System.out.println("d and i " + d + " " + i);

        System.out.println("\nConversion of double to byte.");
        b = (byte) d;
        System.out.println("d and b " + d + " " + b);
    }
}
```

Automatic Type Promotion in Expressions

```
byte a = 40;  
byte b = 50;  
byte c = 100;  
int d = a * b / c;
```

The result of the intermediate term **a * b** easily exceeds the range of either of its **byte** operands.

To handle this kind of problem, Java automatically promotes each **byte**, **short**, or **char** operand to **int** when evaluating an expression.

As useful as the automatic promotions are, they can cause confusing compile-time errors. For example, this seemingly correct code causes a problem:

```
byte b = 50;
```

```
b = b * 2; // Error! Cannot assign an int to a byte!
```

The code is attempting to store $50 * 2$, a perfectly valid **byte** value, back into a **byte** variable.

However, because the operands were automatically promoted to **int** when the expression was evaluated, the result has also been promoted to **int**.

Thus, the result of the expression is now of type **int**, which cannot be assigned to a **byte** without the use of a cast.

In cases where you understand the consequences of overflow, you should use an explicit cast, such as

```
byte b = 50;  
b = (byte) (b * 2);
```

which yields the correct value of 100.

The Type Promotion Rules

Java defines several *type promotion* rules that apply to expressions. They are as follows: First, all **byte**, **short**, and **char** values are promoted to **int**, as just described. Then, if one operand is a **long**, the whole expression is promoted to **long**. If one operand is a **float**, the entire expression is promoted to **float**. If any of the operands are **double**, the result is **double**.

```
class Promote {  
    public static void main(String args[]) {  
        byte b = 42;  
        char c = 'a';  
        short s = 1024;  
        int i = 50000;  
        float f = 5.67f;  
        double d = .1234;  
        double result = (f * b) + (i / c) - (d * s);  
        System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));  
        System.out.println("result = " + result);  
    }  
}
```

Arrays

An *array* is a group of like-typed variables that are referred to by a common name. Arrays of any type can be created and may have one or more dimensions. A specific element in an array is accessed by its index. Arrays offer a convenient means of grouping related information.

One-Dimensional Arrays

A *one-dimensional array* is, essentially, a list of like-typed variables.

To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is

```
type var-name [ ];  
int month_days [ ] ;
```

```
array-var = new type [size];  
month_days = new int[12];
```

```
// Demonstrate a one-dimensional array.
class Array {
public static void main(String args[]) {
    int month_days[];
    month_days = new int[12];
    month_days[0] = 31;
    month_days[1] = 28;
    month_days[2] = 31;
    month_days[3] = 30;
    month_days[4] = 31;
    month_days[5] = 30;
    month_days[6] = 31;
    month_days[7] = 31;
    month_days[8] = 30;
    month_days[9] = 31;
    month_days[10] = 30;
    month_days[11] = 31;
    System.out.println("April has " + month_days[3] + " days.");
}
}
```

```
// An improved version of the previous
program.
class AutoArray {
public static void main(String args[]) {
int month_days[] = { 31, 28, 31, 30, 31, 30,
31, 31, 30, 31, 30, 31 };
System.out.println("April      has      "      +
month_days[3] + " days.");
}
}
```

```
// Average an array of values.  
class Average {  
public static void main(String args[]) {  
double nums[] = {10.1, 11.2, 12.3, 13.4, 14.5};  
double result = 0;  
int i;  
for(i=0; i<5; i++)  
result = result + nums[i];  
System.out.println("Average is " + result / 5);  
}  
}
```

Multidimensional Arrays: `int twoD[][] = new int[4][5];`

```
class TwoDArray {  
    public static void main(String args[]) {  
        int twoD[][]= new int[4][5];  
        int i, j, k = 0;  
        for(i=0; i<4; i++)  
            for(j=0; j<5; j++) {  
                twoD[i][j] = k;  
                k++;  
            }  
        for(i=0; i<4; i++) {  
            for(j=0; j<5; j++)  
                System.out.print(twoD[i][j] + " ");  
            System.out.println();  
        }  
    }  
}
```

Alternative Array Declaration Syntax

There is a second form that may be used to declare an array:

type[] var-name;

```
int al[] = new int[3];
```

```
int[] a2 = new int[3];
```

The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];
```

```
char[][] twod2 = new char[3][4];
```

```
int[] nums, nums2, nums3; // create three arrays
```

```
int nums[], nums2[], nums3[]; // create three arrays
```

Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra.

The following table lists the arithmetic operators:

Operator	Result
+	Addition (also unary plus)
−	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
− =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
− −	Decrement


```
class BasicMath {  
    public static void main(String args[]) {  
  
        System.out.println("Integer Arithmetic");  
        int a = 1 + 1;  
        int b = a * 3;  
        int c = b / 4;  
        int d = c - a;  
        int e = -d;  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
        System.out.println("d = " + d);  
        System.out.println("e = " + e);  
    }  
}
```

```
// Demonstrate the % operator.  
class Modulus {  
public static void main(String args[]) {  
    int x = 42;  
    double y = 42.25;  
    System.out.println("x mod 10 = " + x % 10);  
    System.out.println("y mod 10 = " + y % 10);  
}  
}
```

When you run this program, you will get the following output:

```
x mod 10 = 2  
y mod 10 = 2.25
```

Arithmetic Compound Assignment Operators

`a = a + 4;`

In Java, you can rewrite this statement as shown here:

`a += 4;`

`a = a % 2;`

`a %= 2;`

```
// Demonstrate several assignment operators.
class OpEquals {
public static void main(String args[]) {
int a = 1;
int b = 2;
int c = 3;
a += 5;
b *= 4;
c += a * b;
c %= 6;
System.out.println("a = " + a);
System.out.println("b = " + b);
System.out.println("c = " + c);
}
}
```

The output of this program is shown here:

a = 6, b = 8, c = 3

Increment and Decrement

The ++ and the -- are Java's increment and decrement operators.

The increment operator increases its operand by one. The decrement operator decreases its operand by one. For example, this statement:

```
x = x + 1;
```

can be rewritten like this by use of the increment operator:

```
x++;
```

Similarly, this statement:

```
x = x - 1;
```

```
x --
```

```
x = 42;
```

```
y = ++x;
```

The following program demonstrates the increment operator.

```
// Demonstrate ++.
class IncDec {
public static void main(String args[]) {
int a = 1;
int b = 2;
int c;
int d;
c = ++b;
d = a++;
c++;
System.out.println("a = " + a);
System.out.println("b = " + b);
System.out.println("c = " + c);
System.out.println("d = " + d);
}
}
```

The output of this program follows:

```
a = 2
b = 3
c = 4
d = 1
```

The Bitwise Operators

- Java defines several *bitwise operators* that can be applied to the integer types: **long**, **int**, **short**, **char**, and **byte**.
- These operators act upon the individual bits of their operands.

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

The Bitwise Logical Operators

The bitwise logical operators are **&**, **|**, **^**, and **~**.

A	B	A B	A & B	A ^ B	~A
0	0	0	0	0	1
1	0	1	0	1	0
0	1	1	0	1	1
1	1	1	1	0	0

The Bitwise NOT

Also called the *bitwise complement*, the unary NOT operator, $\sim$, inverts all of the bits of its operand.

For example, the number 42, which has the following bit pattern:
00101010 becomes 11010101 after the NOT operator is applied.

The Bitwise AND

The AND operator, $\&$, produces a 1 bit if both operands are also 1.
A zero is produced in all other cases. Here is an example:

00101010	42
$\&00001111$	15
00001010	10

The Bitwise OR

The OR operator, `|`, combines bits such that if either of the bits in the operands is a 1, then the resultant bit is a 1, as shown here:

00101010	42
00001111	15
00101111	47

The Bitwise XOR

00101010	42
^ 00001111	15
00100101	37

Using the Bitwise Logical Operators

```
class BitLogic {
public static void main(String args[]) {
String binary[] = {"0000", "0001", "0010", "0011", "0100", "0101", "0110",
"0111", "1000", "1001", "1010", "1011", "1100", "1101", "1110", "1111"};
int a = 3; // 0 + 2 + 1 or 0011 in binary
int b = 6; // 4 + 2 + 0 or 0110 in binary
int c = a | b;
int d = a & b;
int e = a ^ b;
int f = (~a & b) | (a & ~b);
int g = ~a & 0x0f;
System.out.println(" a = " + binary[a]);
System.out.println(" b = " + binary[b]);
System.out.println(" a|b = " + binary[c]);
System.out.println(" a&b = " + binary[d]);
System.out.println(" a^b = " + binary[e]);
System.out.println(" ~a&b|a&~b = " + binary[f]);
System.out.println(" ~a = " + binary[g]);
}
}
```

$$\begin{aligned}a &= 0011 \\b &= 0110 \\a \mid b &= 0111 \\a \&b &= 0010 \\a \wedge b &= 0101 \\\sim a \&b \mid a \&\sim b &= 0101 \\\sim a &= 1100\end{aligned}$$

The Left Shift

The left shift operator, `<<`, shifts all of the bits in a value to the left a specified number of times. It has this general form:

value << num

// Left shifting a byte value.

```
class ByteShift {  
    public static void main(String args[]) {  
        byte a = 64, b;  
        int i;  
        i = a << 2;  
        b = (byte) (a << 2);  
        System.out.println("Original value of a: " + a);  
        System.out.println("i and b: " + i + " " + b);  
    }  
}
```

Original value of a: 64

i and b: 256 0

```
// Left shifting as a quick way to multiply by 2.
class MultByTwo {
public static void main(String args[]) {
int i;
int num = 0x02;
for(i=0; i<4; i++) {
num = num << 1;
System.out.println(num) ;
}
}
}
```

The program generates the following output:

4

8

16

32

The Right Shift

The right shift operator, `>>`, shifts all of the bits in a value to the right a specified number of times. Its general form is shown here:

value >> num

```
int a = 32;
```

```
a = a >> 2; // a now contains 8
```

```
int a = 35;
```

```
a = a >> 2; // a contains 8
```

```
11111000 -8
```

```
>> 1
```

```
11111100 -4
```


The Unsigned Right Shift

Java's unsigned, shiftright operator, >>>, which always shifts zeros into the high-order bit.

```
int a = -1;
```

```
a = a >>> 24;
```

Here is the same operation in binary form to further illustrate what is happening:

11111111 11111111 11111111 11111111 -1 in binary as an int

>>>24

00000000 00000000 00000000 11111111 255 in binary as an int

Bitwise Operator Compound Assignments

```
a = a >> 4;
```

```
a >>= 4;
```

```
a = a | b;
```

```
a |= b;
```

```
class OpBitEquals {
```

```
public static void main(String args[]) {
```

```
int a = 1;
```

```
int b = 2;
```

```
int c = 3;
```

```
a |= 4;
```

```
b >>= 1;
```

```
c <<= 1;
```

```
a ^= c;
```

```
System.out.println("a = " + a);
```

```
System.out.println("b = " + b);
```

```
System.out.println("c = " + c); }}
```

Relational Operators

The *relational operators* determine the relationship that one operand has to the other.

Specifically, they determine equality and ordering.

result produced by a relational operator is a boolean value

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

```
int a = 4;
```

```
int b = 1;
```

```
boolean c = a < b;
```

```
int done;
```

```
if(!done)... // Valid in C/C++
```

```
if(done)... // but not in Java.
```

```
if(done == 0)... // This is Java-style.
```

```
if(done != 0)...
```

Boolean Logical Operators

operate only on **boolean** operands. All of the binary logical operators combine two **boolean** values to form a resultant **boolean** value.

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

```
// Demonstrate the boolean logical operators.
class BoolLogic {
public static void main(String args[]) {
boolean a = true;
boolean b = false;
boolean c = a | b;
boolean d = a & b; boolean e = a ^ b;
boolean f = (!a & b) | (a & !b);
boolean g = !a;
System.out.println(" a = " + a);
System.out.println(" b = " + b);
System.out.println(" a|b = " + c);
System.out.println(" a&b = " + d);
System.out.println(" a^b = " + e);
System.out.println("!a&b|a&!b = " + f);
System.out.println(" !a = " + g);
}
}
```

```
    a = true
    b = false
    a | b = true
    a & b = false
    a ^ b = true
    !a & b | a & !b = true
    !a = false
```


The Assignment Operator

var = expression;

int x, y, z;

x = y = z = 100; // set x, y, and z to 100

The ? Operator

Java includes a special *ternary* (three-way) *operator* that

expression1 ? expression2 : expression3

ratio = denom == 0 ? 0 : num / denom;

```
// Demonstrate ?.
class Ternary {
public static void main(String args[]) {
    int i, k;
    i = 10;
    k = i < 0 ? -i : i; // get absolute value of i
    System.out.print("Absolute value of ");
    System.out.println(i + " is " + k);
    i = -10;
    k = i < 0 ? -i : i; // get absolute value of i
    System.out.print("Absolute value of ");
    System.out.println(i + " is " + k);
}
}
```

The output:

Absolute value of 10 is 10

Absolute value of -10 is 10

Operator Precedence

In binary operations, the order of evaluation is left to right (except for assignment, which evaluates right to left).

Highest						
++ (postfix)	-- (postfix)					
++ (prefix)	-- (prefix)	~	!	+ (unary)	- (unary)	(type-cast)
*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
&&						
?:						
->						
=	op=					
Lowest			Dept. of CSE(AI&ML)-AS			

Using Parentheses

Parentheses raise the precedence of the operations that are inside them.

a >> b + 3

This expression first adds 3 to **b** and then shifts **a** right by that result. That is, this expression can be rewritten using redundant parentheses like this:

a >> (b + 3)

However, if you want to first shift **a** right by **b** positions and then add 3 to that result, you will need to parenthesize the expression like this:

(a >> b) + 3

In addition to altering the normal precedence of an operator, parentheses can sometimes be used to help clarify the meaning of an expression

$a \mid 4 + c \gg b \& 7$
 $(a \mid ((4 + c) \gg b) \& 7)$

Control Statements

A programming language uses *control* statements to cause the flow of execution to advance and branch based on changes to the state of a program.

Java's program control statements can be put into the following categories:

- Selection,
- Iteration, and
- Jump.

Selection statements allow your program to choose different paths of execution based upon the outcome of an expression or the state of a variable.

Iteration statements enable program execution to repeat one or more statements (that is, iteration statements form loops).

Jump statements allow your program to execute in a nonlinear fashion.

Java's Selection Statements

Java supports two selection statements: **if** and **switch**.

```
if (condition) statement1;
```

```
else statement2;
```

```
int a, b;
```

```
//...
```

```
if(a < b) a = 0;
```

```
else b = 0;
```


If you want to include more statements, you'll need to create a block, as in this fragment:

```
int bytesAvailable;  
// ...  
if (bytesAvailable > 0) {  
    processData();  
    bytesAvailable -= n;  
}  
  
else  
    waitForMoreData();
```

Nested ifs

```
if(i == 10) {
```

```
  if(j < 20)
```

```
    a = b;
```

```
    if(k > 100)
```

```
      c = d; // this if is
```

```
      else a = c; // associated with this else
```

```
    }
```

```
  else a = d; // this else refers to if(i == 10)
```

The if-else-if Ladder

A common programming construct that is based upon a sequence of nested **ifs** is the *if-elseif*

ladder. It looks like this:

```
if(condition)  
    statement;  
else if(condition)  
    statement;  
else if(condition)  
    statement;  
.  
.  
.else  
    statement;
```

```
// Demonstrate if-else-if statements.
class IfElse {
public static void main(String args[]) {
int month = 4; // April
String season;
if(month == 12 || month == 1 || month == 2)
season = "Winter";
else if(month == 3 || month == 4 || month == 5)
season = "Spring";
else if(month == 6 || month == 7 || month == 8)
season = "Summer";
else if(month == 9 || month == 10 || month == 11)
season = "Autumn";
else
season = "Bogus Month";
System.out.println("April is in the " + season + ".");
}
}
```

switch

The **switch** statement is Java's multiway branch statement. It provides an easy way to dispatch execution to different parts of your code based on the value of an expression

the general form of a **switch** statement:

```
switch (expression) {  
  case value1:  
    // statement sequence  
    break;  
  case value2:  
    // statement sequence  
    break;  
  .  
  .  
  .  
  case valueN :  
    // statement sequence  
    break;  
  default:  
    // default statement sequence  
}
```

```
// A simple example of the switch.
class SampleSwitch {
public static void main(String args[]) {
for(int i=0; i<6; i++)
switch(i) {
case 0:
System.out.println("i is zero.");
break;
case 1:
System.out.println("i is one.");
break;
case 2:
System.out.println("i is two.");
break;
case 3:
System.out.println("i is three.");
break;
default:
System.out.println("i is greater than 3.");
}
}
}
```

The output produced by this program

```
i is zero.
i is one.
i is two.
i is three.
i is greater than 3.
i is greater than 3.
```

```
class MissingBreak {
public static void main(String args[]) {
for(int i=0; i<12; i++)
switch(i) {
case 0:
case 1:
case 2:
case 3:
case 4:
System.out.println("i is less than 5");
break;
case 5:
case 6:
case 7:
case 8:
case 9:
System.out.println("i is less than 10");
break;
default:
System.out.println("i is 10 or more");
}
}
}
```

This program generates the following output:

```
i is less than 5  
i is less than 5  
i is less than 5  
i is less than 5  
i is less than 5  
i is less than 10  
i is less than 10  
i is less than 10  
i is less than 10  
i is less than 10  
i is 10 or more  
i is 10 or more
```



```
// An improved version of the season
program.
class Switch {
public static void main(String args[]) {
int month = 4;
String season;
switch (month) {
case 12:
case 1:
case 2:
season = "Winter";
break;
case 3:
case 4:
case 5:
season = "Spring";
break;
case 6:
case 7:
case 8:
season = "Summer";
break;
```

```
case 9:  
case 10:  
case 11:  
season = "Autumn";  
break;  
default:  
season = "Bogus Month";  
}  
System.out.println("April is in the " + season + ".");  
}  
}
```

```
// Use a string to control a switch statement.
class StringSwitch {
public static void main(String args[]) {
String str = "two";
switch(str) {
case "one":
System.out.println("one");
break;
case "two":
System.out.println("two");
break;
case "three":
System.out.println("three");
break;
default:
System.out.println("no match");
break;
}}}
```

Nested switch Statements

You can use a **switch** as part of the statement sequence of an outer **switch**. This is called a *nested switch*.

```
switch(count) {  
  case 1:  
    switch(target) { // nested switch  
      case 0:  
        System.out.println("target is zero");  
        break;  
      case 1: // no conflicts with outer switch  
        System.out.println("target is one");  
        break;  
    }  
    break;  
  case 2: // ...
```

Iteration Statements

Java's iteration statements are **for**, **while**, and **do-while**.

These statements create what we commonly call *loops*.

As you probably know, a loop repeatedly executes the same set of instructions until a termination condition is met

while

The **while** loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true. Here is its general form:

```
while(condition) {  
    // body of loop  
}
```

```
// Demonstrate the while loop.  
class While {  
public static void main(String args[])  
{  
int n = 10;  
while(n > 0) {  
System.out.println("tick " + n);  
n--;  
}  
}  
}
```

```
int a = 10, b = 20;  
while(a > b)  
System.out.println("This will not be displayed");
```

the body of the loop will not execute even once if the condition is false to begin with

```
// The target of a loop can be empty.  
class NoBody {  
public static void main(String args[]) {  
int i, j;  
i = 100;  
j = 200;  
// find midpoint between i and j  
while(++i < --j); // no body in this loop  
System.out.println("Midpoint is " + i);  
}  
}
```

This program finds the midpoint between **i** and **j**.

It generates the following output:

```
Midpoint is 150
```


do-while

As you just saw, if the conditional expression controlling a **while** loop is initially false, then the body of the loop will not be executed at all.

However, sometimes it is desirable to execute the body of a loop at least once, even if the conditional expression is false to begin with.

```
do {  
    // body of loop  
}
```

```
// Demonstrate the do-while loop.  
class DoWhile {  
public static void main(String args[]) {  
    int n = 10;  
    do {  
        System.out.println("tick " + n);  
        n--;  
    } while(n > 0);  
}  
}
```

The for loop

The **for** loop is a looping construct which can execute a set of instructions a specified number of times. It's a counter controlled loop.

Syntax:

```
for (<initialization>; <loop condition>; <increment expression>) {  
    <loop body>  
}
```

Example:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Printing Numbers from 1 to 10");  
        for (int count = 1; count <= 10; count++)  
        {  
            System.out.println(count);  
        }  
    }  
}
```

Declaring Loop Control Variables Inside the for Loop

- Often the variable that controls a **for** loop is only needed for the purposes of the loop and is not used elsewhere.
- When this is the case, it is possible to declare the variable inside the initialization portion of the **for**.

```
class ForTick {  
    public static void main(String args[]) {  
        for(int n=10; n>0; n--)  
            System.out.println("tick " + n);  
    }  
}
```

- When you declare a variable inside a **for** loop, there is one important point to remember: the scope of that variable ends when the **for** statement does

Using the Comma

There will be times when you will want to include more than one statement in the initialization and iteration portions of the **for** loop.

```
class Comma {  
    public static void main(String args[]) {  
        int a, b;  
        for(a=1, b=4; a<b; a++, b--) { System.out.println("a = " + a);  
            System.out.println("b = " + b);  
        }  
    }  
}
```

Some for Loop Variations

```
boolean done = false;  
for(int i=1; !done; i++) {  
    // ...  
    if(interrupted()) done = true;  
}
```

In this example, the **for** loop continues to run until the **boolean** variable **done** is set to **true**. It does not test the value of **i**.

// Parts of the for loop can be empty.

```
class ForVar {  
    public static void main(String args[]) { int i;  
        boolean done = false; i = 0;  
        for( ; !done; ) {  
            System.out.println("i is " + i);  
            if(i == 10) done = true;  
            i++;  
        }  
    }  
}
```

Here, the initialization and iteration expressions have been moved out of the **for**. Thus, parts of the **for** are empty

- Here is one more **for** loop variation. You can intentionally create an infinite loop (a loop that never terminates) if you leave all three parts of the **for** empty.
- For example:

```
for( ; ; ) {  
    // ...  
}
```

This loop will run forever because there is no condition under which it will terminate.

The For-Each Version of the for Loop

- The general form of the for-each version of the **for** is shown here:

```
for(type itr-var : collection)  
    statement-block
```

```
class ForEach {  
    public static void main(String args[]) {  
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };  
  
        int sum = 0;  
  
        for(int x : nums) {  
            sum += x;  
        }  
  
        System.out.println("Summation: " + sum);  
    }  
}
```

For example, this program sums only the first five elements of **nums**:

```
class ForEach2 {  
    public static void main(String args[]) {  
        int sum = 0;  
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };  
        // use for to display and sum the values  
  
        for(int x : nums) {  
            sum += x;  
            if(x == 5)  
                break; // stop the loop when 5 is obtained  
        }  
        System.out.println("Summation of first 5 elements: " + sum);  
    }  
}
```

Java program to search given key element

```
class Search {  
    public static void main(String args[]) {  
        int nums[] = { 6, 8, 3, 7, 5, 6, 1, 4 };  
        int val = 5;  
        boolean found = false;  
  
        for(int x : nums) {  
            if(x == val) {  
                found = true; break;  
            }  
        }  
  
        if(found)  
            System.out.println("Value found!");  
    }  
}
```

Nested Loops

- Like all other programming languages, Java allows loops to be nested.
- That is, one loop may be inside another. For example, here is a program that nests **for**

```
loops: class Nested {  
  
    public static void main(String args[]) {  
  
        int i, j;  
        for(i=0; i<10; i++) {  
            for(j=i; j<10; j++)  
  
                System.out.print("."); System.out.println();  
        }  
    }  
}
```

Jump Statements

- Java supports three jump statements: **break**, **continue**, and **return**. These statements transfer control to another part of your program.

The break statement

- The **break** statement transfers control out of the enclosing loop (for, while, do or switch statement).
- You use a **break** statement when you want to jump immediately to the statement following the enclosing control structure.
- You can also provide a loop with a label, and then use the label in your **break** statement.
- The label name is optional, and is usually only used when you wish to terminate the outermost loop in a series of nested loops.

Example for break:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Numbers 1 - 10");  
  
        for (int i = 1;; ++i) {  
            if (i == 11)  
                break;  
  
            System.out.println(i + "\t");  
        }  
    }  
}
```

Syntax:

break; // the unlabeled form

break <label>; // the labeled form

The **continue** statement

- A **continue** statement stops the iteration of a loop (while, do or for) and causes execution to resume at the top of the nearest enclosing loop.
- You use a **continue** statement when you do not want to execute the remaining statements in the loop, but you do not want to exit the loop itself.
- You can also provide a loop with a label and then use the label in your **continue** statement.
- The label name is optional, and is usually only used when you wish to return to the outermost loop in a series of nested loops.

Example for continue:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Odd Numbers");  
  
        for (int i = 1; i <= 10; ++i) {  
            if (i % 2 == 0)  
                continue;  
  
            System.out.println(i + "\t");  
        }  
    }  
}
```

Syntax:

continue; // the unlabeled form

continue <label>; // the labeled form

The return statement

- The **return** statement exits from the current method, and control flow returns to where the method was invoked.

Syntax:

The **return** statement has two forms:

One that returns a value

return val;

One that doesn't returns a value

return;

Example:

```
public class Example {  
    public static void main(String[] args) {  
        int res = sum(10, 20);  
        System.out.println(res);  
    }  
    private static int sum(int a, int b) {  
        return (a + b);  
    }  
}
```